

Event-Nodes, Texture-Nodes, and Causal Structure: From State Intervention to Event-Historical Mechanistic Explanation

Flyxion

December 2025

1 Introduction

Recent advances in mechanistic interpretability have emphasized causal reasoning as the criterion separating genuine understanding from descriptive inspection. Interventions, counterfactuals, and causal abstractions offer a disciplined way to test whether a hypothesized algorithm is truly implemented by a system rather than merely correlated with its behavior.

However, standard causalmechanistic approaches remain implicitly state-centric. Interventions are typically defined as instantaneous value substitutions, and irreversibility enters only through accidental information loss (e.g. noise or ablation). This essay argues that a minimal extension of the framework is required to account for history-sensitive structure, irreversible commitments, and the emergence of constraints that persist beyond any single state.

To this end, we introduce two complementary causal primitives: *event-nodes* and *texture-nodes*. These notions allow causal graphs to represent not only values and pathways, but also the historical processes and structural textures that shape future behavior.

—

2 Causal Graphs and State-Based Intervention

Let a system be represented as a directed acyclic graph

$$\mathcal{G} = (V, E),$$

where vertices V correspond to internal variables and edges E represent causal influence. Traditional interventions replace the value of a node $v \in V$ with an externally supplied value v' , breaking incoming edges while preserving outgoing ones.

This formalism excels at identifying mediating variables and testing necessity or sufficiency within a fixed causal structure. However, it treats causal nodes as temporally atomic: values are swapped, tested, and restored without regard to how those values were produced.

Such a framework can identify *paths of influence*, but not *paths of becoming*.

—

3 Event-Nodes

Definition 1 (Event-Node). *An event-node is a causal node whose primary explanatory role is not the value it holds, but the irreversible constraint it imposes on the system’s future admissible states.*

Formally, an event-node $e \in \mathcal{E}$ induces a transition

$$\mathcal{F}_{t+1} = \mathcal{F}_t \cap \Phi_e,$$

where $\Phi_e \subseteq \mathcal{S}$ is a constraint set defining which future states remain reachable.

Unlike state variables, event-nodes cannot be fully characterized by their instantaneous values. Their causal significance lies in the *reduction of future possibility space*. An intervention on an event-node is therefore not merely a substitution, but a commitment.

This allows irreversibility to be treated as structural rather than accidental. Even if all information is preserved, an event-node may permanently foreclose certain counterfactual trajectories.

4 Texture-Nodes

Definition 2 (Texture-Node). *A texture-node is a causal node that parametrizes the generative structure of variation within a system, shaping how local perturbations propagate without directly determining outcomes.*

Texture-nodes do not encode discrete decisions or constraints. Instead, they modulate *how* behavior is generated. Their role is analogous to material or shader parameters in a three-dimensional mesh: they do not alter topology, but they determine surface character, continuity, and response to deformation.

Let $\mathcal{N} : \mathcal{S} \times \Theta \rightarrow \mathcal{S}$ be a noise or variation generator. A texture-node corresponds to a parameter vector $\theta \in \Theta$ such that

$$s_{t+1} = \mathcal{N}(s_t, \theta).$$

Crucially, θ may itself be the result of prior events. Thus texture-nodes often record ratios, gradients, or directional biases accumulated over time.

5 Analogy: Meshes, Materials, and Generative Noise

Consider a geometric mesh in a modeling environment. The mesh defines topology; materials define appearance. Changing a material does not move vertices, yet it radically alters how light, texture, and noise interact with the surface.

In this analogy:

- State-nodes correspond to vertex positions.
- Event-nodes correspond to irreversible topology edits.

- Texture-nodes correspond to material parameters.
- Noise generators correspond to procedural variation functions.

A texture-node selects a generator function whose output depends on a ratio or vector parameter:

$$\theta = \frac{\alpha}{\beta},$$

where α and β may represent accumulated gradients, pressures, or historical frequencies. The system records θ not as a symbolic label, but as a bias shaping future dynamics.

Thus, two systems with identical topology and instantaneous states may diverge due to differing textures: the same perturbation produces different outcomes because the generative field differs.

—

6 Counterfactuals Over Futures

Traditional counterfactuals ask:

What would the output have been if this variable had taken a different value?

Event- and texture-nodes enable a stronger question:

What futures would still be reachable if this event or texture had been different?

Let $\mathcal{R}(s)$ denote the reachable future set from state s . An event-level counterfactual compares $\mathcal{R}(s \mid e)$ to $\mathcal{R}(s \mid e')$. A texture-level counterfactual compares the shape of $\mathcal{R}(s)$ under different generative parameters θ .

This reframes semantic compatibility: a substitution fails not because a value is wrong, but because it presupposes a future space that no longer exists.

—

7 Causal Abstraction Revisited

A causal abstraction is traditionally validated by interventional equivalence. With event- and texture-nodes, abstraction gains an additional requirement:

Definition 3 (Temporal Coherence). *A causal abstraction is temporally coherent if it preserves both interventional outcomes and the irreversibility structure of the underlying system.*

An abstraction that predicts reversibility where the system exhibits irreversible settlement is incomplete, even if its inputoutput behavior matches locally.

This explains why multiple abstractions may coexist: each compresses structure differently, preserving certain textures while discarding others.

—

8 Failure, Collapse, and Mixture

Failures under intervention are not noise; they are diagnostic. A collapse into incoherence indicates that a texture-node or event-node essential to generative stability has been disrupted.

Systems that exhibit patchwork behavior under intervention are best understood as mixtures of event-histories and textures rather than single algorithms. Their apparent inconsistency reflects genuine structural plurality.

9 What It Means to Understand

Understanding is not achieved by control alone, nor by prediction in isolation. It consists in reconstructing:

- the event-nodes that constrain futures,
- the texture-nodes that shape variation,
- and the causal graph that organizes both.

Such understanding is necessarily partial. Texture is lossy under abstraction, and event-history cannot be fully replayed. But these limitations are principled, not failures.

10 Conclusion

By extending causal graphs with event-nodes and texture-nodes, mechanistic interpretability gains the expressive power to describe irreversible structure, generative variation, and historical constraint without abandoning causal rigor.

The result is a framework in which mechanisms are not merely paths through a graph, but histories inscribed into the future where what a system *can no longer do* is as explanatory as what it does.

11 Necessary Conditions for Texture-Nodes to Become Event-Nodes

This section derives conditions under which a variable that initially functions as a *texture-node*—a parameter modulating a generative field without directly constraining the admissible futures—must be reclassified as an *event-node*, i.e. as a node whose causal significance is the irreversible restriction of future reachable states.

11.1 Preliminaries: Reachability and Generative Fields

Let $\mathcal{S}$ be the system state space and let $\mathcal{R}_T(s)$ denote the set of states reachable from $s \in \mathcal{S}$ within a horizon $T \in \mathbb{N}$. We assume a one-step update law of the form

$$s_{t+1} = F(s_t; \theta_t), \quad (1)$$

where $\theta_t \in \Theta$ is a parameter vector. When θ_t is a texture-node, its intended role is to modulate the generative field (e.g. the noise, smoothness, anisotropy, sparsity, or routing biases) without imposing hard, history-settling constraints.

To make the modulation explicit, we may decompose

$$F(s; \theta) = G(s) + H(s; \theta), \quad (2)$$

where G is the “baseline” dynamics and H is the texture-controlled component (which may be stochastic or deterministic).

Definition 4 (Texture-Equivalence of Futures). *Two parameter values $\theta, \theta' \in \Theta$ are texture-equivalent at horizon T for state s if*

$$\mathcal{R}_T(s \mid \theta) = \mathcal{R}_T(s \mid \theta'). \quad (3)$$

They are approximately texture-equivalent if the reachable sets are equal up to a specified tolerance (e.g. Hausdorff distance or probability mass distance under a stochastic transition kernel).

Under pure texture modulation, changes in θ may alter the *measure* over futures (which trajectories are likely), while preserving the *support* of futures (which trajectories are possible). The distinction between measure and support is the fulcrum of the event/texture boundary.

11.2 Support Restriction Criterion

Assume the system admits a stochastic kernel representation

$$s_{t+1} \sim K_{\theta_t}(\cdot \mid s_t), \quad (4)$$

so that for any measurable set $A \subseteq \mathcal{S}$,

$$\mathbb{P}(s_{t+1} \in A \mid s_t) = \int_A K_{\theta_t}(ds' \mid s_t). \quad (5)$$

Definition 5 (Support of the One-Step Kernel). *The support of $K_{\theta}(\cdot \mid s)$ is the smallest closed set $\text{supp } K_{\theta}(\cdot \mid s) \subseteq \mathcal{S}$ with probability one.*

Proposition 1 (Necessary Condition: Support Change). *If a texture-node value θ is to remain a pure texture-node (i.e. not an event-node), then for all relevant states s and horizons T , changes in θ may alter likelihoods but must not remove futures from the reachable support. In particular, it is*

necessary that

$$\text{supp } K_\theta(\cdot \mid s) = \text{supp } K_{\theta'}(\cdot \mid s) \quad \text{for all } s \text{ in the regime of interest.} \quad (6)$$

Proof. If $\text{supp } K_\theta(\cdot \mid s) \neq \text{supp } K_{\theta'}(\cdot \mid s)$, then there exists a set A such that one parameter value assigns nonzero probability to transitioning into A while the other assigns zero probability. This implies a strict inclusion between one-step reachable sets, and therefore a strict inclusion between T -step reachable sets for some T (at worst $T = 1$). Hence the parameter change removes admissible futures and functions as a constraint on reachability, which is the defining role of an event-node. $\square$

Intuitively, a texture parameter may act like a material selection that changes surface appearance, but if it changes the mesh’s admissible deformations (e.g. turns a soft body into a rigid body that cannot bend), then it has become a structural operator and thus an event.

11.3 Irreversibility Criterion via Hysteresis

Even if a parameter does not change one-step support, it may still be event-like if its *update law* introduces hysteresis or path dependence.

Assume the texture parameter evolves according to

$$\theta_{t+1} = U(\theta_t, s_t), \quad (7)$$

where U may include thresholding, clipping, quantization, or saturation.

Definition 6 (Hysteretic Texture Update). *The update U is hysteretic on a set $\Omega \subseteq \Theta \times \mathcal{S}$ if there exist two histories $(\theta_0, s_0, \dots, s_t)$ and $(\theta'_0, s'_0, \dots, s'_t)$ such that $(\theta_t, s_t) = (\theta'_t, s'_t)$ but $\theta_{t+1} \neq \theta'_{t+1}$.*

Proposition 2 (Necessary Condition: No Hysteresis Under State Equivalence). *If θ is to remain a pure texture-node, it is necessary that its update law be non-hysteretic under the state equivalences used in causal analysis. Concretely, whenever the causal framework treats two situations as equivalent (e.g. identical activations at the intervention boundary), the texture update must agree:*

$$(\theta_t, s_t) \sim (\theta'_t, s'_t) \implies U(\theta_t, s_t) = U(\theta'_t, s'_t). \quad (8)$$

Proof. If two situations are treated as equivalent by the causal testing protocol but yield different θ_{t+1} , then the future reachable sets diverge purely due to untracked historical information. In that case θ encodes irreducible history, and interventions that only set instantaneous values are insufficient to restore counterfactual symmetry. The node is therefore functioning as a history-settling operator, i.e. an event-node. $\square$

Hysteresis corresponds to a material parameter whose value depends on *how* it was reached, not merely its current numeric value. In procedural shading terms, this is the difference between a parameter that selects a noise seed and a parameter that has been permanently baked into a texture map.

11.4 Thresholding and Phase Boundary Condition

Many systems contain sharp boundaries where a continuously varying texture parameter induces a qualitative shift in reachable structure.

Suppose there exists a scalar summary $\rho(\theta)$ (for example, a ratio of two accumulated quantities) such that the dynamics effectively changes regime when $\rho(\theta)$ crosses a threshold ρ_* :

$$F(s; \theta) = \begin{cases} F_-(s; \theta), & \rho(\theta) < \rho_*, \\ F_+(s; \theta), & \rho(\theta) \geq \rho_*. \end{cases} \quad (9)$$

Proposition 3 (Necessary Condition: No Regime-Switching Under Small Perturbations). *For θ to remain texture-like on a regime of interest, it is necessary that small perturbations in θ not induce regime switching that changes reachable support. In particular, for all (s, θ) in the regime and all sufficiently small δ ,*

$$\mathcal{R}_T(s \mid \theta) \approx \mathcal{R}_T(s \mid \theta + \delta) \quad (10)$$

in support, not merely in probability.

Proof. If arbitrarily small perturbations can flip the system between distinct transition regimes whose reachable supports differ, then the parameter acts as a gate selecting among different future constraint families. Such gating is structural and thus event-like. $\square$

This is the minimal formal statement of a “phase boundary.” Once a texture parameter gates between distinct dynamical laws, it ceases to be mere texture.

11.5 Recording Criterion: When a Parameter Becomes a Commitment

Your guiding analogy includes that the texture generator depends on a recorded ratio that functions as a vector parameter for a variable in question. Let

$$\theta_t = \Psi\left(\frac{a_t}{b_t}\right), \quad (11)$$

where a_t, b_t are accumulated quantities (counts, energies, gradients, frequencies) and Ψ maps the ratio into a vector parameter (direction, scale, anisotropy, routing preference).

If a_t and b_t are merely summaries that can be recomputed from current state, then θ_t remains texture-like. If instead a_t and b_t encode irrecoverable history, then θ_t inherits event-like character.

Proposition 4 (Necessary Condition: Reconstructibility of the Recorded Ratio). *If θ_t is computed from a recorded ratio $\frac{a_t}{b_t}$, then a necessary condition for θ to remain a texture-node is that the ratio be reconstructible from the instantaneous causal boundary used in interventions. That is, there must exist a function Ξ such that*

$$\frac{a_t}{b_t} = \Xi(s_t) \quad (12)$$

on the regime of interest (or, in the stochastic case, that its conditional distribution depends only on s_t).

Proof. If the ratio cannot be reconstructed from the intervention boundary, then two states treated as identical under the causal protocol may still carry different latent ratios, hence different θ_t , hence different future supports or constraints. The parameter then encodes untracked history that cannot be erased by state-level substitution, making it a commitment variable, i.e. event-like. $\square$

In the mesh analogy, reconstructibility corresponds to whether the material choice is a pure function of current shader inputs, versus a baked selection made earlier in the pipeline that cannot be inferred from the final rendered surface alone.

11.6 Summary: Minimal Necessary Conditions

The boundary between texture-nodes and event-nodes is not metaphysical. It is determined by reachability, reconstructibility, and irreversibility. A node initially treated as a texture-node must be promoted to an event-node whenever any of the following necessary conditions fail in the regime of interest:

1. **Support invariance:** changing the parameter does not change the support of the transition kernel (no futures become impossible).
2. **No hysteresis:** the parameter update does not depend on untracked history under the equivalences used by intervention tests.
3. **No regime gating:** small changes in the parameter do not switch the system between distinct dynamical regimes with different reachable supports.
4. **Reconstructible recording:** any recorded ratio or summary that drives the parameter can be reconstructed from the causal boundary state (or its conditional distribution depends only on that boundary state).

When these conditions hold, the parameter behaves as a genuine texture: it reshapes variation without settling futures. When any condition fails, the parameter is no longer merely a material choice; it becomes a structural edit to the system’s admissible counterfactual space, and therefore functions as an event-node.

12 Sufficiency Conditions and the Event–Texture Phase Boundary

The previous section derived *necessary* conditions under which a texture-node must be promoted to an event-node. We now derive complementary *sufficiency* conditions: conditions under which a variable can be treated as a genuine texture-node without loss of causal adequacy within a given regime. Together, these results define a sharp phase boundary between texture and event structure.

12.1 Texture Sufficiency via Future-Space Equivalence

Let $\mathcal{R}_\infty(s \mid \theta)$ denote the infinite-horizon reachable future set from state s under parameter θ . While practical analysis often works at finite horizons, the conceptual distinction between texture and event is asymptotic: event-nodes settle futures permanently.

Definition 7 (Future-Space Equivalence Class). *Two parameter values $\theta, \theta' \in \Theta$ are said to lie in the same future-space equivalence class on a regime $\Omega \subseteq \mathcal{S}$ if*

$$\mathcal{R}_\infty(s \mid \theta) = \mathcal{R}_\infty(s \mid \theta') \quad \text{for all } s \in \Omega. \quad (13)$$

Proposition 5 (Sufficiency Condition: Asymptotic Equivalence). *If a parameter θ varies only within a single future-space equivalence class on the regime of interest, then θ can be treated as a texture-node for all causal abstractions restricted to that regime.*

Proof. By hypothesis, variations in θ do not alter the set of admissible infinite-horizon futures. Consequently, no irreversible commitments are introduced by changing θ , and all counterfactual trajectories remain reachable under replay. Thus θ does not impose structural constraints and functions purely as a modulator of trajectory selection within a fixed future space, which is precisely the role of a texture-node. $\square$

This formalizes the intuitive notion that texture governs *how* futures are sampled, not *which* futures exist.

12.2 Local Reversibility and Retexturing

A second sufficiency condition concerns reversibility under intervention.

Definition 8 (Local Retexturability). *A parameter θ is locally retexturable at state s if, for any θ' in the parameter domain, there exists a finite sequence of state-level interventions $\{I_k\}$ such that*

$$(s, \theta) \xrightarrow{I_1, \dots, I_n} (s', \theta') \quad (14)$$

with $\mathcal{R}_\infty(s') = \mathcal{R}_\infty(s)$.

Proposition 6 (Sufficiency Condition: Retexturability). *If a parameter is locally retexturable throughout the regime of interest, then it cannot function as an event-node within that regime.*

Proof. Local retexturability guarantees that any value of θ can be imposed without altering the infinite-horizon future space. Thus no parameter setting constitutes a point of no return. Since event-nodes are defined by irreversible constraint of futures, a locally retexturable parameter lacks the defining property of an event-node. $\square$

In the mesh analogy, this corresponds to a material parameter that can always be swapped at render time, never requiring a topology rebuild or geometry bake.

12.3 Texture Stability Under Causal Abstraction

A further sufficiency condition concerns abstraction itself.

Definition 9 (Abstraction-Stable Texture). *A parameter θ is abstraction-stable if, for every valid causal abstraction $\mathcal{A}$ of the system on the regime of interest, the image of θ under $\mathcal{A}$ does not correspond to an event-node in the abstracted graph.*

Proposition 7 (Sufficiency Condition: Abstraction Stability). *If a parameter is abstraction-stable across all abstractions admissible for the intended explanatory task, then treating it as a texture-node is sufficient for mechanistic understanding at that level.*

Proof. If no abstraction relevant to the explanatory task requires θ to be represented as an irreversible constraint, then θ plays no essential role in determining the abstract system’s commitment structure. Representing θ as a texture-node preserves all explanatory distinctions made at that level, and promoting it to an event-node would add structure without explanatory gain. $\square$

This result makes explicit that texture versus event is not an absolute classification, but a level-relative one.

12.4 Phase Boundary Summary

Combining necessary and sufficiency conditions, we obtain a precise phase boundary:

Remark 1 (Event–Texture Phase Boundary). *A causal variable functions as a texture-node on a regime of interest if and only if:*

1. *its variation preserves the support of reachable futures,*
2. *it exhibits no hysteresis under the equivalence relations used by intervention protocols,*
3. *it does not gate regime changes under small perturbations,*
4. *any recorded summaries driving it are reconstructible at the causal boundary,*
5. *its values lie within a single future-space equivalence class,*
6. *and it is locally retexturable without altering future reachability.*

Violation of any of these conditions forces reclassification as an event-node for that regime.

The distinction is thus neither metaphysical nor merely descriptive. It is a formal consequence of how a system partitions its future. Texture governs variation within a future; event governs which futures remain possible.

This phase boundary provides a principled basis for extending causal mechanistic analysis beyond instantaneous state substitution to systems whose behavior is shaped by accumulated structure, recorded ratios, and irreversible settlement.

13 Connection to Spherpops Calculus

The distinction between event-nodes and texture-nodes admits a direct and natural correspondence with the primitives of Spherpops calculus. This section makes that correspondence explicit and shows that the extended causal framework developed above can be seen as a semantic realization of Spherpops operational rules.

13.1 Event-Nodes as Pop Operations

In Spherepop calculus, a POP is defined as an irreversible operation that eliminates a subset of admissible futures. This is not a change of state in the extensional sense, but a settlement of possibility.

Event-nodes coincide exactly with this notion.

Formally, let $\mathcal{H}_t$ denote the admissible future set at time t . A Spherepop POP induces a transition

$$\mathcal{H}_{t+1} = \mathcal{H}_t \setminus \Delta, \quad (15)$$

for some nonempty exclusion set Δ .

This is identical to the action of an event-node as defined earlier:

$$\mathcal{F}_{t+1} = \mathcal{F}_t \cap \Phi_e, \quad (16)$$

with $\Phi_e = \mathcal{S} \setminus \Delta$.

Thus:

$$\textbf{Event-node} \longleftrightarrow \textbf{Spherepop Pop}$$

Both are characterized by:

- irreversibility,
- future-space restriction,
- and independence from instantaneous representational value.

Crucially, neither requires destructive noise or loss of information. The irreversibility is structural, not epistemic.

13.2 Texture-Nodes as Spherepop Bind Structure

Spherepop calculus distinguishes BIND operations from POPs. A BIND introduces ordering, dependency, or constraint relations between future actions without eliminating options outright.

Texture-nodes implement exactly this role at the causalmechanistic level.

A texture-node:

- reshapes the generative field,
- biases which futures are likely,
- and introduces anisotropy or smoothness,

while preserving the support of admissible futures.

This matches the operational role of BIND:

$$\mathcal{H}_{t+1} = \mathcal{H}_t, \quad \mu_{t+1} \neq \mu_t, \quad (17)$$

where μ_t is a measure or ordering over $\mathcal{H}_t$.

Thus:

$$\textbf{Texture-node} \longleftrightarrow \textbf{Spherepop Bind}$$

The noise-generator analogy fits precisely here. A texture-node selects or modulates a generator function whose parameters are ratios or vectors recorded from prior activity. These ratios do not settle futures, but they bias how future trajectories are drawn.

13.3 Recorded Ratios and Spherepop History

In Spherepop calculus, objects and relations exist only insofar as they are introduced by events. Identity is historical, not extensional.

The recorded ratios driving texture-nodes correspond to *derived historical summaries* in Spherepop: quantities accumulated from prior pops and binds that shape subsequent dynamics without constituting new pops themselves.

Let $r_t = a_t/b_t$ be a recorded ratio influencing a texture-node parameter $\theta_t = \Psi(r_t)$. In Spherepop terms:

- a_t and b_t are functions of the event history,
- r_t is a historical invariant,
- θ_t is a derived binding parameter.

When r_t is reconstructible from the current causal boundary, the binding remains soft and replayable. When it is not, the ratio encodes untracked history, and the texture-node must be promoted to an event-node. In Spherepop terms, a BIND has hardened into a POP.

This precisely mirrors the necessary-condition results derived earlier.

13.4 Collapse and Spherepop Collapse

Spherepop calculus defines COLLAPSE as the erasure of historical differentiation in order to restore flexibility. This operation discards event records while preserving optionality.

In the causal framework, collapse corresponds to:

- resetting or marginalizing event-nodes,
- erasing recorded ratios,
- restoring symmetry between counterfactual futures.

Importantly, collapse is not the inverse of a pop. It does not recover lost futures; it removes the record that distinguished them. This aligns with the observation that even after collapse, some futures may remain inaccessible if the underlying structure has been irreversibly altered.

13.5 Causal Abstraction as Spherepop Replay

Spherepop calculus emphasizes replay: the idea that meaning and identity arise from the reproducibility of event histories.

Causal abstraction in the extended mechanistic framework can be understood as *replay equivalence*:

Two systems are abstractly equivalent if replaying corresponding sequences of pops and binds yields the same admissible future structure.

Interchange interventions correspond to replaying alternative event sequences under controlled substitutions. Abstraction validity requires that replay at the abstract level preserves both:

- inputoutput counterfactuals, and
- the structure of irreversible settlement.

This reframes causal abstraction not as pattern matching between graphs, but as compatibility between event algebras.

13.6 Summary

The correspondence can be summarized succinctly:

Causal Framework	Spherepop Calculus
Event-node	POP
Texture-node	BIND
Support restriction	Loss of admissible futures
Recorded ratio	Historical invariant
Retexturing	Soft binding
Irreversible promotion	Bind $\rightarrow$ Pop
Collapse	COLLAPSE
Causal abstraction	Replay equivalence

Viewed this way, the extended causalmechanistic framework is not merely compatible with Spherepop calculus; it is a concrete instantiation of its event-historical semantics. Event-nodes and texture-nodes provide the missing bridge between abstract operational rules and empirically testable causal interventions, grounding Spherepops notions of irreversibility, commitment, and replay in mechanistic analysis.

14 Categorical and Sheaf-Theoretic Interpretation

The distinction between event-nodes and texture-nodes admits a precise reformulation in categorical and sheaf-theoretic terms. Doing so clarifies both the limits of state-based causal analysis and the sense in which event-historical structure constitutes genuine additional information rather than mere

annotation. In this section, event-nodes are interpreted as categorical operations that alter the base category of admissible futures, while texture-nodes are interpreted as internal parameterizations living in the fibers of a sheaf over that base.

14.1 The Category of Admissible Futures

Let $\mathcal{C}_t$ denote a category whose objects are admissible future continuations from time t , and whose morphisms represent valid transitions between such continuations. Composition corresponds to temporal concatenation. At any given moment, mechanistic analysis typically assumes a fixed $\mathcal{C}_t$, implicitly treating the space of possible futures as stable under intervention.

An event-node, however, does not merely select an object or morphism within $\mathcal{C}_t$. Instead, it induces a functor

$$E : \mathcal{C}_t \longrightarrow \mathcal{C}_{t+1}$$

that is not essentially surjective. Objects in $\mathcal{C}_t$ that are not in the image of E correspond to futures that have been irrevocably excluded. Irreversibility, in this formulation, is categorical non-invertibility: there exists no functor E^{-1} whose composition with E restores the original category up to equivalence.

This makes precise the sense in which event-nodes are not state updates but changes in the ambient space of possibility. They act on the *indexing category* of futures itself.

14.2 Texture-Nodes as Fiberwise Structure

Texture-nodes, by contrast, do not alter the base category $\mathcal{C}_t$. They live in additional structure carried over it. To formalize this, consider a sheaf $\mathcal{F}$ over $\mathcal{C}_t$, where to each object (future) x we associate a space $\mathcal{F}(x)$ of admissible generative parameters. Sections of this sheaf correspond to coherent assignments of texture across compatible futures.

A texture-node corresponds to the choice of a section (or local section) of $\mathcal{F}$. Changing a texture-node alters which section is selected, but does not alter the underlying objects of $\mathcal{C}_t$. In other words, the support of futures remains fixed; only their internal geometry is reshaped.

This sheaf-theoretic perspective makes clear why texture-nodes modulate likelihoods, smoothness, or anisotropy without settling outcomes. They change the internal structure of fibers while preserving the base.

14.3 Recorded Ratios as Sections with Memory

The recorded ratios discussed earlier acquire a natural interpretation in this setting. A ratio such as $r_t = a_t/b_t$ determines a parameter $\theta_t = \Psi(r_t)$, which in turn selects a local section of $\mathcal{F}$. If the ratio is reconstructible from the current causal boundary, then the section depends only on local data and defines a genuine sheaf section.

If, however, the ratio depends on global historical information that cannot be recovered from the boundary state, then the would-be section fails the sheaf gluing condition. It is no longer determined by compatible local data, but by a choice that depends on how those locals were assembled. In categorical terms, the assignment ceases to be natural.

This failure of naturality corresponds exactly to the promotion of a texture-node to an event-node. The parameter can no longer be treated as living in the fibers of a fixed base category; instead, it must be reified as an operation on the base itself.

14.4 Event Promotion as Base Change

The transition from texture-node to event-node can therefore be understood as a change of base. What was previously modeled as a sheaf $\mathcal{F} \rightarrow \mathcal{C}_t$ must now be modeled as a different base category $\mathcal{C}_{t+1}$ with its own sheaf of textures. The original sheaf does not extend across the event, because the objects it was defined over no longer exist.

This clarifies why no amount of state-level intervention can undo certain effects: once the base category has changed, interventions that assume the old base are ill-typed. Counterfactual substitutions fail not because values are incorrect, but because they presuppose morphisms that are no longer present.

14.5 Causal Abstraction as Functorial Projection

Causal abstraction can now be reformulated as the search for a functor

$$A : \mathcal{C}_t \longrightarrow \mathcal{D}$$

from the category of admissible futures to a simpler category $\mathcal{D}$ representing an abstract algorithm. Validity of abstraction requires that A commute, up to natural isomorphism, with the event functors that define the systems irreversible structure.

In particular, if an abstraction collapses distinctions that are preserved by event functors predicting reversibility where the concrete system exhibits categorical collapse then it fails to be a faithful representation of the systems temporal structure. This provides a categorical restatement of the temporal coherence criterion introduced earlier.

14.6 Spherepop Reinterpreted Categorically

Within this formalism, Spherepop calculus emerges as an algebra of base changes and sheaf reconfigurations. POP operations correspond to non-invertible endofunctors on the category of futures. BIND operations correspond to changes of section within a fixed sheaf. COLLAPSE corresponds to a quotienting or truncation operation that identifies histories while preserving the base category.

Replay, finally, corresponds to functorial equivalence: two histories are meaningfully the same if they induce equivalent categories of admissible futures together with equivalent sheaves of texture.

14.7 Interpretive Payoff

The categorical and sheaf-theoretic formulation sharpens the central thesis of this work. Mechanistic understanding is not merely about identifying variables or paths, but about determining which aspects of a system live in the fibers and which act on the base. Texture is fiberwise and replayable;

event structure is base-altering and irreversible. Confusing the two leads either to spurious claims of control or to the false expectation that all mechanisms can be undone by sufficiently clever intervention.

By making this distinction explicit, the framework provides a principled bridge between causal experimentation, event-historical semantics, and abstract mathematical structure, without inflating the ontology beyond what mechanistic analysis strictly requires.

15 Final Synthesis: Mechanism, History, and Texture

The aim of this essay has been to reconcile two perspectives that are often treated as incompatible: the causal-mechanistic program, with its emphasis on intervention and counterfactual control, and an event-historical view in which irreversibility, commitment, and accumulated structure play an essential explanatory role. The introduction of event-nodes and texture-nodes provides the minimal conceptual vocabulary required to unify these perspectives without weakening either.

At the level of causal experimentation, state-based interventions remain indispensable. They allow one to isolate mediators, trace information flow, and test hypotheses about algorithmic structure. However, taken alone, such interventions implicitly assume that the space of admissible futures is fixed and replayable. This assumption holds only within regimes where all relevant structure is texture-like where variation reshapes likelihoods but does not settle possibilities.

Event-nodes mark the precise point at which this assumption fails. They are not distinguished by dramatic representational change, nor by loss of information, but by a categorical alteration of what futures remain reachable. Their effects cannot be undone by clever substitution because they operate at the level of constraints rather than values. In this sense, event-nodes formalize commitment: they explain why certain counterfactuals are no longer meaningful, not merely why they are unlikely.

Texture-nodes occupy the complementary role. They encode how variation is generated, how noise is structured, and how tendencies are biased without foreclosing alternatives. The analogy to material parameters in a generative mesh is exact: texture changes appearance, smoothness, and response, but leaves the underlying topology intact. When texture parameters are derived from recorded ratios or accumulated statistics, they carry history forward in a soft and replayable form. Only when such recording becomes non-reconstructible or gates regime change does texture harden into event.

The necessary and sufficiency conditions derived earlier make this boundary precise. A variable must be promoted to an event-node exactly when it restricts the support of reachable futures, introduces hysteresis under the equivalences assumed by intervention, or encodes history that cannot be erased by state-level replay. Conversely, variables that preserve future-space equivalence, admit retexturing, and remain fiberwise across valid abstractions may be safely treated as texture-nodes within the relevant regime.

The categorical and sheaf-theoretic reformulation clarifies why this distinction is not a matter of modeling preference. Event-nodes act on the base category of admissible futures, while texture-nodes live in sheaves over that base. Promotion from texture to event corresponds to a change of base, not

merely a refinement of parameters. Abstractions that ignore this distinction may succeed locally yet fail to respect the temporal coherence of the system they purport to explain.

Seen through this lens, Spherepop calculus is not an external metaphor but an operational algebra for these distinctions. POP corresponds to the introduction of event-nodes, BIND to the modulation of texture, and COLLAPSE to the controlled erasure of historical differentiation. Causal abstraction becomes replay equivalence: two systems are equivalent insofar as their sequences of events and textures induce equivalent categories of futures.

The resulting picture of mechanistic understanding is neither purely state-based nor irreducibly historical. Understanding consists in determining which aspects of a system are replayable textures and which are irrevocable events, and in constructing abstractions that preserve this distinction. What a system can no longer do is as much a part of its mechanism as what it does.

This synthesis suggests a natural stopping point for mechanistic explanation. Beyond this boundary, further detail does not yield deeper understanding but merely finer texture. Within it, causal intervention, event history, and mathematical structure align to provide a coherent account of how systems compute, commit, and constrain their own futures.

A Appendix: Implementing Spherepop Operators in Forth

This appendix sketches a concrete implementation strategy for Spherepop operators in a small, inspectable Forth system. The design goal is not to build a complete runtime, but to demonstrate that Spherepops event-historical semantics can be realized as a minimal stack language where every operation either (i) records an irreversible event, (ii) introduces replayable structural binding, or (iii) collapses historical differentiation. Forth is particularly well-suited because it makes evaluation order explicit, keeps state small and visible, and encourages dictionary-driven extension.

A.1 Representation Strategy

A Spherepop system is most naturally represented as two coupled structures. The first is a persistent *event log* whose entries are the primitive operations (POP, BIND, MERGE, LINK, COLLAPSE). The second is a derived *present state* computed from replaying some prefix (or a compressed summary) of the log. In practice, a Forth implementation keeps both: the event log provides semantic ground truth, while the derived state provides efficient execution.

We assume a simple model in which objects are integer identifiers. Each object has an associated parent pointer or part-of relation, plus optional metadata. The semantics of part-of can be built as a union-find variant (for merge-like operations) supplemented by additional relations (for link-like operations). The essential Spherepop requirement is that these relations are not treated as axiomatic; they exist only insofar as they are generated by recorded events.

A.2 Forth Data Structures

We implement a small heap using arrays. The details can be varied across Forth systems; the following style is compatible with common ANS Forth conventions.

We represent:

1. a monotonically increasing object counter,
2. an event log with fixed-width records,
3. parent/representative arrays for merge semantics,
4. an optional adjacency list region for link semantics.

The event log records a tag and up to three integer fields. The tag determines interpretation. This deliberately mirrors the idea that the log is authoritative, and the present state is derived.

```
\ -----
\ Core storage sizes
\ -----
10000 CONSTANT MAX-OBJ
20000 CONSTANT MAX-EVT

\ -----
\ Object allocator
\ -----
VARIABLE NEXT-ID

\ -----
\ Union-Find parent + rank
\ -----
CREATE PARENT MAX-OBJ CELLS ALLOT
CREATE RANK    MAX-OBJ CELLS ALLOT

\ -----
\ Event log: (tag a b c) per event
\ tag is a small integer
\ -----
VARIABLE EVT#
CREATE EVT-TAG MAX-EVT CELLS ALLOT
CREATE EVT-A   MAX-EVT CELLS ALLOT
CREATE EVT-B   MAX-EVT CELLS ALLOT
CREATE EVT-C   MAX-EVT CELLS ALLOT
```

A.3 Initialization and Allocation

Initialization clears counters and sets each object to be its own parent.

```
: INIT ( -- )
  0 NEXT-ID !
  0 EVT# !
```

```

;

: NEW ( -- id )
  NEXT-ID @ DUP 1+ NEXT-ID !
  DUP CELLS PARENT + !      \ parent[id] = id
  0 OVER CELLS RANK + !     \ rank[id] = 0
;

```

The word `NEW` implements the creation of a fresh entity. In a strict event-historical reading, creation itself is an event and should be logged. One can therefore define `POP` (below) to include allocation, or define `NEW` as a convenience word that calls `POP` with a distinguished tag.

A.4 Event Logging

We define a helper word that appends a record to the event log.

```

: LOG-EVT ( tag a b c -- )
  EVT# @ DUP
  >R
  CELLS EVT-TAG + !
  R@ CELLS EVT-A   + !
  R@ CELLS EVT-B   + !
  R> CELLS EVT-C   + !
  EVT# @ 1+ EVT# !
;

```

This realizes the fundamental Spharepop thesis that semantics should be sourced from events. Every operator below calls `LOG-EVT`.

A.5 Union-Find Core for Merge

A minimal `MERGE` can be implemented by union-find. This yields an efficient derived part-of structure while the authoritative meaning remains in the log.

```

: FIND ( x -- root )
  DUP CELLS PARENT + @ OVER = IF EXIT THEN
  DUP CELLS PARENT + @ RECURSE
  DUP ROT CELLS PARENT + !
;

: UNION ( x y -- )
  FIND SWAP FIND
  2DUP = IF 2DROP EXIT THEN
  \ union by rank

```

```

2DUP CELLS RANK + @ SWAP CELLS RANK + @
2DUP < IF \ rank[x] < rank[y]
  DROP SWAP \ ensure first has >= rank
ELSE DROP THEN
\ now: xRoot yRoot with rank[x]>=rank[y]
OVER CELLS PARENT + ! \ parent[yRoot] = xRoot
\ if ranks equal, increment rank
2DUP CELLS RANK + @ SWAP CELLS RANK + @ = IF
  OVER CELLS RANK + DUP @ 1+ SWAP !
THEN
2DROP
;

```

A.6 Spherepop Operators as Forth Words

We now define Spherepop operators. The semantics here are intentionally minimal and should be read as a reference architecture rather than a completed theory. The key feature is that every operator both (i) performs a derived-state update and (ii) logs itself as an event for replay.

A.6.1 Pop

A POP is an irreversible event that introduces a new object and thereby changes the future space by committing to a new historical fact: the object exists and can be referred to. One may also interpret POP as a commitment that excludes certain future histories in which the object did not exist.

```

1 CONSTANT TAG-POP

: POP ( -- id )
  NEW DUP 0 0 0
  TAG-POP ROT ROT ROT LOG-EVT
;

```

The particular record fields are unused here; in richer variants, POP may attach provenance or a parent context.

A.6.2 Bind

A BIND introduces ordering or dependency without removing options. In a minimal implementation, BIND can be represented as a constraint edge in a separate structure (e.g. a dependency graph), while leaving the union-find structure untouched. For compactness, we log it and omit derived enforcement; the essential point is the event-historical record.

```

2 CONSTANT TAG-BIND

```

```

: BIND ( a b -- )
  \ meaning: a must precede b, or a constrains b
  0 TAG-BIND ROT ROT 0 LOG-EVT
;

```

A full implementation would maintain a constraint graph and enforce acyclicity or other admissibility criteria. The present definition demonstrates the operational encoding: binding is recorded and therefore participates in replay.

A.6.3 Merge

A MERGE identifies two objects into a common equivalence class. In the derived state, this is union-find; in the log, it is a recorded event.

```

3 CONSTANT TAG-MERGE

: MERGE ( a b -- )
  2DUP UNION
  0 TAG-MERGE ROT ROT 0 LOG-EVT
;

```

A.6.4 Link

A LINK introduces a relation between objects without identifying them. This can be modeled as an adjacency list or as a set of labeled edges. Again we log the event and supply a minimal derived mechanism.

Here we assume a fixed-size edge store. The representation can be improved, but the essential semantics are: link is explicit, replayable, and historically grounded.

```

4 CONSTANT TAG-LINK
100000 CONSTANT MAX-LINK
VARIABLE LNK#
CREATE LNK-FROM MAX-LINK CELLS ALLOT
CREATE LNK-TO   MAX-LINK CELLS ALLOT

: LINK-ADD ( a b -- )
  LNK# @ DUP >R
  R@ CELLS LNK-FROM + !
  R> CELLS LNK-TO   + !
  LNK# @ 1+ LNK# !
;

: LINK ( a b -- )
  2DUP LINK-ADD

```

```

0 TAG-LINK ROT ROT 0 LOG-EVT
;

```

A.6.5 Collapse

A COLLAPSE discards historical differentiation while preserving flexibility. Operationally, collapse is best understood as log compression: replace a detailed prefix of events with a summary that yields the same derived state for the purposes of future evolution, while erasing the fine-grained historical distinctions.

In Forth, the simplest demonstration is to reset the log to a checkpoint while retaining the derived structures. This does not undo the derived constraints; it forgets the path by which they were achieved. More sophisticated collapse can store a digest event that records only invariants.

```

5 CONSTANT TAG-COLLAPSE
VARIABLE EVT-CUT

: CHECKPOINT ( -- )
  EVT# @ EVT-CUT !
;

: COLLAPSE ( -- )
  \ Forget events prior to EVT-CUT by truncation.
  \ Derived state is kept; only historical detail is removed.
  EVT-CUT @ EVT# !
  0 0 0 TAG-COLLAPSE 0 0 0 LOG-EVT
;

```

The call to LOG-EVT after truncation functions as a marker indicating that a collapse occurred. In a richer system, the collapse record would contain a hash of the retained derived state or a canonical summary of invariants.

A.7 Replay and Verification

Replay is central to Spherepop semantics. In this implementation, replay means resetting the derived state and reapplying the event log to reconstruct the present. The following word replays events, interpreting each tag.

```

: RESET-DERIVED ( -- )
  \ Reset union-find for all allocated objects.
  NEXT-ID @ 0 DO
    I DUP CELLS PARENT + !
    0 I CELLS RANK + !
  LOOP
  0 LNK# !

```

```

;

: EVT@ ( i -- tag a b c )
  DUP CELLS EVT-TAG + @
  OVER CELLS EVT-A    + @
  OVER CELLS EVT-B    + @
  SWAP CELLS EVT-C    + @
;

: APPLY ( tag a b c -- )
  \ interpret a single event
  >R >R >R \ stash c b a
  CASE
    TAG-POP OF \ pop: nothing to do; object already allocated in this toy model
      R> R> R> DROP DROP DROP
    ENDOF
    TAG-BIND OF
      R> R> R> DROP DROP DROP
    ENDOF
    TAG-MERGE OF
      R> R> SWAP R> DROP \ a b c -> a b
    UNION
  ENDOF
  TAG-LINK OF
    R> R> SWAP R> DROP
  LINK-ADD
  ENDOF
  TAG-COLLAPSE OF
    R> R> R> DROP DROP DROP
  ENDOF
  ENDCASE
;

: REPLAY ( -- )
  RESET-DERIVED
  EVT# @ 0 DO
    I EVT@ APPLY
  LOOP
;

```

This replay mechanism exhibits the intended separation: the log is the semantic authority and the derived structures are reconstructions. As written, POP does not allocate during replay; a strict

version would log allocation and reallocate in replay, or store object counts as part of a checkpoint event. The point is architectural: there is a canonical history and a derived present.

A.8 Event-Nodes and Texture-Nodes in Forth Terms

Within this implementation, event-nodes correspond to log entries whose replay changes the admissible future space. POP and MERGE are event-like because they introduce new referents or collapse referents irreversibly in the future algebra. COLLAPSE is also event-like, though of a different kind: it changes the space of counterfactual explanation by erasing distinctions.

Texture-nodes correspond to parameters that alter generative behavior without changing what is admissible in principle. In Forth, these are naturally modeled as variables that affect the interpretation of subsequent events or the stochasticity/heuristics by which new events are proposed, while leaving replay equivalence intact. The most faithful implementation strategy is therefore to treat texture as fiberwise metadata: recorded ratios, seeds, and generator parameters that can be changed without altering the set of admissible histories, so long as they satisfy the reconstructibility and support-preservation conditions derived earlier.

A.9 Remarks on Safety and Minimality

This appendix intentionally avoids dynamic code generation, self-modifying definitions, or arbitrary memory writes. Forth makes such techniques possible, but Spherepops semantic commitment is stronger when the only authoritative structure is the event log and its replay. The minimal implementation therefore prefers explicit records and replayable interpreters, because these preserve the intended distinction between structural history and derived present state.

In this sense, Forth serves as an existence proof: Spherepop operators can be realized in a compact, transparent substrate where mechanistic inspection is straightforward and where the semantics of irreversibility are expressed as literal log structure rather than as implicit side effects.

B Appendix: Implementing Spherepop Operators in Haskell

This appendix provides a Haskell implementation sketch for Spherepop operators analogous in spirit to the preceding Forth appendix. The guiding design constraint is that the systems semantics be *event-sourced*: the event log is authoritative, while the present state is a derived object obtained by replay or by replay-equivalent compression. The implementation therefore separates (i) the type of events, (ii) an interpreter (replay function) from events to state, and (iii) a small API of Spherepop operators that append to the log.

The code below is intentionally minimal and intended as a reference architecture. It is designed to be readable, auditable, and compatible with pure functional reasoning. Performance and storage layout can be improved later without changing the semantic core.

B.1 Core Types: Objects, Events, and Histories

We represent objects as integer identifiers. A history is a list of events. The event type is explicit and closed; this is the key to keeping semantics inspectable.

```
{-# LANGUAGE DeriveGeneric #-}

module Spherepop where

import GHC.Generics (Generic)
import Data.Map.Strict (Map)
import qualified Data.Map.Strict as M
import Data.Set (Set)
import qualified Data.Set as S

-- | Object identifiers.
newtype Obj = Obj { unObj :: Int }
    deriving (Eq, Ord, Show, Generic)

-- | Spherepop events. The log is authoritative.
data Event
    = Pop Obj
    | Bind Obj Obj
    | Merge Obj Obj
    | Link Obj Obj
    | Collapse Checkpoint
    deriving (Eq, Show, Generic)

-- | A checkpoint can be as small as a log index plus an optional digest.
data Checkpoint = Checkpoint
    { cutIndex :: Int
    , digest    :: Maybe String
    } deriving (Eq, Show, Generic)

-- | A history is a sequence of events.
newtype History = History { unHistory :: [Event] }
    deriving (Eq, Show, Generic)
```

B.2 Derived State

A derived state is an interpretation of the history. We implement a minimal state consisting of a union-find style representative map for merges, a set of links, and an optional set of binding constraints. Bind constraints are recorded as data; enforcing them (e.g. checking acyclicity) can be

layered on later.

```
-- | Representative mapping for Merge semantics.
-- Invariant: rep[x] is the chosen representative for x's class.
type Rep = Map Obj Obj

-- | Link relation as a set of directed edges.
type Links = Set (Obj, Obj)

-- | Bind constraints as a set of directed edges (ordering or dependency).
type Binds = Set (Obj, Obj)

data State = State
  { reps    :: Rep
  , links   :: Links
  , binds   :: Binds
  , next    :: Int
  } deriving (Eq, Show)

emptyState :: State
emptyState = State
  { reps    = M.empty
  , links   = S.empty
  , binds   = S.empty
  , next    = 0
  }
```

The field `next` provides fresh identifiers. In a strict event-historical semantics, allocation should itself be recorded as an event. We do so by ensuring that `pop` constructs the fresh object and records it explicitly.

B.3 Union-Find Operations

For minimality, we implement a simple representative-chasing `find` operation with path compression. Union chooses the representative deterministically.

```
findRep :: Rep -> Obj -> Obj
findRep r x =
  case M.lookup x r of
    Nothing -> x
    Just p   ->
      let root = findRep r p
      in root
```

```

compress :: Rep -> Obj -> Rep
compress r x =
  case M.lookup x r of
    Nothing -> r
    Just p   ->
      let root = findRep r p
      in M.insert x root r

unionRep :: Rep -> Obj -> Obj -> Rep
unionRep r a b =
  let ra = findRep r a
      rb = findRep r b
  in if ra == rb
    then r
    else
      -- deterministically pick smaller Obj as representative
      let rep = min ra rb
          sub  = max ra rb
          r'   = M.insert sub rep r
      in compress (compress r' a) b

```

B.4 Replay Semantics: Interpreting Events

Replay is the semantic heart: given a history, compute the derived state. This function is pure and therefore mechanically checkable. In particular, any two histories that replay to the same state are equivalent at the chosen level of derived representation.

```

apply :: Event -> State -> State
apply ev st =
  case ev of
    Pop o ->
      -- Ensure the object exists in reps map (optional) and advance allocation.
      st { reps = M.insertWith (\_ old -> old) o o (reps st) }

    Bind a b ->
      st { binds = S.insert (a,b) (binds st) }

    Merge a b ->
      st { reps = unionRep (reps st) a b }

    Link a b ->

```

```

    st { links = S.insert (a,b) (links st) }

Collapse cp ->
  -- Collapse is handled at the history level; apply is a no-op here.
  -- We still record its existence in the log, and replay interprets it
  -- as a truncation marker when requested.
  st

replay :: History -> State
replay (History evs) =
  foldl (flip apply) emptyState evs

```

The `Collapse` event is intentionally treated as a semantic instruction about the history rather than as a state mutation. This mirrors the idea that collapse erases historical differentiation while preserving the derived present.

B.5 Collapse as Log Compression

We implement collapse as truncation to a checkpoint index followed by insertion of a `Collapse` marker. More sophisticated implementations can replace the truncated prefix with a digest event that stores canonical invariants.

```

collapse :: Checkpoint -> History -> History
collapse cp (History evs) =
  let k    = max 0 (min (cutIndex cp) (length evs))
      evs' = take k evs
  in History (evs' ++ [Collapse cp])

```

If one prefers that collapse preserve derived state exactly while discarding path detail, then the checkpoint should be taken at a point where replayed state is already acceptable as a canonical summary. Alternatively, one can store a digest that commits to a particular derived state and treat the digest as an event-node in the sense of the main text.

B.6 Spherepop Operators as an API

We now define a small API that maintains a history and a derived state in a single structure. In practice, the derived state is optional because it can always be recomputed from replay, but keeping it avoids repeated folds.

```

data World = World
  { hist  :: History
  , state :: State
  } deriving (Eq, Show)

```

```

emptyWorld :: World
emptyWorld = World (History []) emptyState

append :: Event -> World -> World
append ev (World (History evs) st) =
  let st' = apply ev st
  in World (History (evs ++ [ev])) st'

-- | POP allocates a new object and records it.
pop :: World -> (Obj, World)
pop (World h st) =
  let o    = Obj (next st)
      st'  = st { next = next st + 1 }
      w'   = append (Pop o) (World h st')
  in (o, w')

bind :: Obj -> Obj -> World -> World
bind a b = append (Bind a b)

merge :: Obj -> Obj -> World -> World
merge a b = append (Merge a b)

link :: Obj -> Obj -> World -> World
link a b = append (Link a b)

-- | COLLAPSE compresses history at the log level, then rebuilds derived state.
collapseWorld :: Checkpoint -> World -> World
collapseWorld cp (World h _) =
  let h'  = collapse cp h
      st'  = replay h'
  in World h' st'

```

This API makes explicit what the event-historical semantics demands: all meaningful changes are events, and the present is derived. The only non-eventful change above is `next`, the allocator counter; this is an implementation detail. If allocation must itself be replayable from events, then the allocator counter should be computed from the history (e.g. by counting `Pop` events), and `State.next` can be removed.

B.7 Event-Nodes and Texture-Nodes as Types

The main body of the essay distinguishes event-nodes from texture-nodes. In the Haskell setting, the distinction becomes a typing discipline: events are logged and replayable, while textures are parameters used by interpreters that modulate behavior without changing admissible futures.

A minimal way to represent this is to parameterize the interpreter by a texture record. Texture does not appear in the history; it appears in the semantics of how events are interpreted or proposed.

```
data Texture = Texture
  { noiseSeed  :: Int
  , ratioParam :: Double
  } deriving (Eq, Show)

-- An interpreter that is allowed to consult texture.
applyWithTexture :: Texture -> Event -> State -> State
applyWithTexture _ ev st =
  apply ev st
```

In the simplest case, texture is irrelevant to replay, and therefore has no effect on admissible futures; it merely affects how new events are generated. In richer systems, texture may modulate stochastic proposals, heuristic selection of merges, or prioritization of links. The necessary and sufficient conditions derived earlier then become properties of `applyWithTexture`: if varying `Texture` changes reachability support or introduces hysteresis not reconstructible at the causal boundary, the texture must be promoted into the event log as an event-node.

B.8 A Small Worked Example

```
example :: World
example =
  let (a,w1) = pop emptyWorld
      (b,w2) = pop w1
      w3      = link a b w2
      w4      = merge a b w3
      w5      = bind a b w4
      w6      = collapseWorld (Checkpoint 3 Nothing) w5
  in w6
```

The example demonstrates the intended separation. Events are appended, replay reconstructs, and collapse truncates history while preserving a derived present. The meaning of the system is the history; the state is its current projection.

B.9 Remarks on Extensibility

This Haskell design is deliberately conservative. It favors explicit event types and a single interpreter rather than embedding Spheredpop semantics into an opaque monad. Nevertheless, it scales naturally. One can add typed variants, invariants, and proof-carrying summaries by refining the `Event` type and the replay function. One can also add categorical structure by interpreting histories as morphisms in a free category generated by events, with `replay` providing a functor into a concrete state category.

Finally, one can implement sheaf-like texture structure by treating **Texture** as a fiberwise assignment whose gluing conditions correspond to reconstructibility constraints.

In all cases, the central Spherepop principle is preserved: the authoritative meaning is an event history, and the operational present is a derived, replayable projection of that history.

C Appendix: Implementing Spherepop Operators in Racket

This appendix presents a Racket implementation sketch of Spherepop operators parallel to the Forth and Haskell appendices. The architectural constraint is again event-sourcing: the event log is the semantic authority, while any “current state” is a derived projection obtained by interpreting (replaying) that log. Racket is well-suited for this because it provides algebraic data via structs, persistent maps/sets, and a macro system for building a compact surface syntax without changing the underlying semantics.

C.1 Core Representation

We represent objects as integers. Events are represented as transparent structs so that they are printable and inspectable. A history is a list of events. The derived state is a record containing representative mappings (for merge), link edges, bind constraints, and an allocation counter. As in the other appendices, the allocation counter may be eliminated if one insists that allocation be computed solely from the history; it is kept here to keep the operational API simple.

```
#lang racket

(require racket/set
         racket/match)

;; -----
;; Events
;; -----

(struct pop      (id)      #:transparent)
(struct bind     (a b)     #:transparent)
(struct merge    (a b)     #:transparent)
(struct link     (a b)     #:transparent)
(struct collapse (cut digest) #:transparent)

;; A history is a list of events.
;; (listof (U pop bind merge link collapse))

;; -----
;; Derived State
```

```
;; -----
```

```
(struct state (reps links binds next) #:transparent)
```

```
(define empty-state  
  (state (hash) (set) (set) 0))
```

C.2 Union-Find Style Representatives

We implement a representative map for merges. This is a minimal union-find variant expressed in purely functional style. The representative of an object is obtained by chasing pointers in a hash map. Path compression can be added, but the minimal version suffices for semantic clarity.

```
(define (find-rep reps x)  
  (define p (hash-ref reps x #f))  
  (if (or (not p) (= p x))  
      x  
      (find-rep reps p)))  
  
(define (union-rep reps a b)  
  (define ra (find-rep reps a))  
  (define rb (find-rep reps b))  
  (cond [(= ra rb) reps]  
        [else  
         (define rep (min ra rb))  
         (define sub (max ra rb))  
         (hash-set reps sub rep)]))
```

C.3 Replay Semantics

Replay interprets a history as a fold over events. This gives a canonical meaning to a history: two histories are equivalent at the derived level if replay yields the same state (or the same invariants of state).

Collapse is treated as a history-level operation rather than a state mutation. In the replay function we therefore ignore collapse events, except insofar as they are used as markers in separate compression procedures.

```
(define (apply-event ev st)  
  (match ev  
    [(pop id)  
     (define reps (state-reps st))  
     (define reps* (if (hash-has-key? reps id)  
                        reps
```



```

                (hash-set reps id id)))
  (struct-copy state st [reps reps*]))

[(bind a b)
 (struct-copy state st [binds (set-add (state-binds st) (cons a b))])]

[(merge a b)
 (struct-copy state st [reps (union-rep (state-reps st) a b))]]

[(link a b)
 (struct-copy state st [links (set-add (state-links st) (cons a b))])]

[(collapse _ _)
 st]))

(define (replay hist)
  (for/fold ([st empty-state])
    ([ev (in-list hist)])
    (apply-event ev st)))

```

C.4 History-Level Collapse (Compression)

Collapse is defined as erasing historical differentiation while preserving the derived present. Operationally, the minimal implementation truncates the log to a cut index and appends a collapse marker. A more faithful implementation would replace the truncated prefix with a digest event encoding canonical invariants. The digest field is included for that purpose, even if unused here.

```

(define (collapse-history hist cut [digest #f])
  (define cut* (max 0 (min cut (length hist))))
  (append (take hist cut*)
    (list (collapse cut* digest))))

```

C.5 World API: Event-Sourcing with Derived Cache

As in the other appendices, we provide a `world` record containing both history and a cached derived state. Every operator appends an event and updates the cache by interpreting that single event. Collapse rebuilds the cache by replaying the compressed history, reflecting the idea that collapse is a history transformation rather than an ordinary state transition.

```

(struct world (hist st) #:transparent)

(define empty-world
  (world '() empty-state))

```

```

(define (append-event w ev)
  (define hist* (append (world-hist w) (list ev)))
  (define st* (apply-event ev (world-st w)))
  (world hist* st*))

(define (POP w)
  (define id (state-next (world-st w)))
  (define st* (struct-copy state (world-st w) [next (add1 id)]))
  (define w* (world (world-hist w) st*))
  (values id (append-event w* (pop id))))

(define (BIND w a b)
  (append-event w (bind a b)))

(define (MERGE w a b)
  (append-event w (merge a b)))

(define (LINK w a b)
  (append-event w (link a b)))

(define (COLLAPSE w cut [digest #f])
  (define hist* (collapse-history (world-hist w) cut digest))
  (world hist* (replay hist*)))

```

C.6 Texture Parameters as Fiberwise Semantics

The main essay distinguishes event-nodes from texture-nodes. In this Racket realization, the minimal way to express that distinction is to keep textures out of the history and instead treat them as parameters of interpretation or of event proposal. That is, textures are not logged events; they are fiberwise metadata that modulate generative behavior without changing admissible futures, unless they violate the support-preservation and reconstructibility conditions derived in the main text.

One can express this by extending the interpreter to consult a texture record when applying events, or by making event-proposal functions depend on textures.

```

(struct texture (seed ratio) #:transparent)

(define default-texture
  (texture 0 1.0))

(define (apply-event/texture tex ev st)
  ;; In the simplest form, texture does not affect replay semantics at all.

```

```
;; More elaborate versions can allow texture to bias heuristic choices,
;; while remaining fiberwise if it preserves future support.
(apply-event ev st))
```

If texture begins to gate regime changes, introduce hysteresis not reconstructible from the causal boundary, or change the reachable support, then it can no longer be treated as a mere parameter; it must be promoted into the event log as an event-node, for instance by introducing a new event constructor such as `(set-texture tex)` and replaying it as part of the authoritative history.

C.7 A Small Worked Example

```
(define (example)
  (define-values (a w1) (POP empty-world))
  (define-values (b w2) (POP w1))
  (define w3 (LINK w2 a b))
  (define w4 (MERGE w3 a b))
  (define w5 (BIND w4 a b))
  (define w6 (COLLAPSE w5 3))
  w6)

(example)
```

The example illustrates the intended semantics. All meaning-bearing actions are events appended to a log. Replay reconstructs the present. Collapse compresses history while preserving a derived state. The design is deliberately explicit: every important object is a plain struct, and every semantic transformation is a total function over histories.

C.8 Macro Surface Syntax (Optional)

Racket permits a compact surface syntax without changing the semantic model. One can introduce syntactic forms that expand into calls to `POP`, `BIND`, `MERGE`, `LINK`, and `COLLAPSE`. This provides an ergonomic “Spheredpop script” language while retaining the same replay semantics. The macro layer is intentionally omitted here to keep the appendix focused on the semantic core, but the presence of transparent event structs makes such a layer straightforward: a macro expands into event-appending calls, and the log remains the canonical source of truth.

In this sense, the Racket implementation serves the same purpose as the Forth and Haskell sketches: it demonstrates that Spheredpop operators are not merely philosophical primitives but can be realized as a small, inspectable, replayable event calculus in a conventional programming environment.